



JAVA 8 STREAM API

Prepared By
Bhaskara Sai Chitturi

Java Stream API

In modern Java programming, working with large data sets and collections is a common task. Whether you're filtering, transforming, or analyzing data, the process often requires looping over collections and manually applying operations. This is where Java's Stream API comes in, making these operations much simpler and more readable.

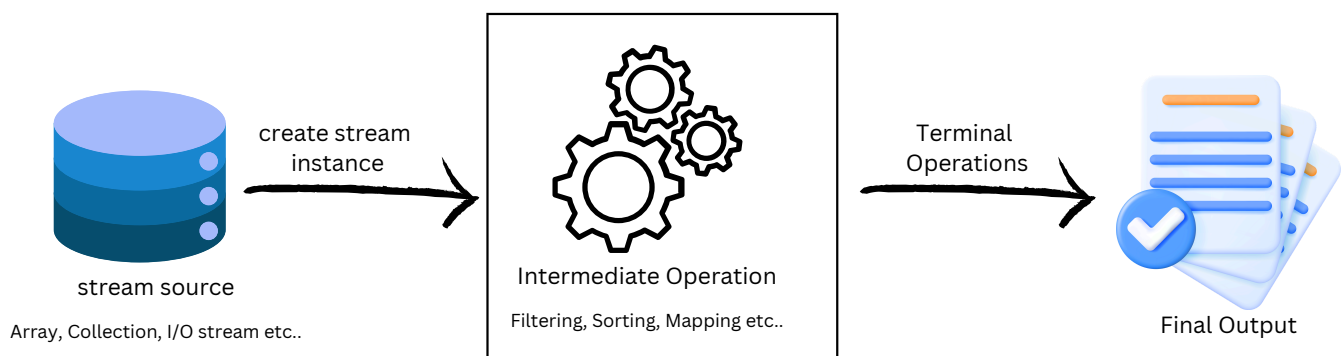
Java Stream API introduced in Java 8, Stream API used to process, iterate, perform operations on collections, until Java 7 we have for and for Each to process the operations on collections.



A Stream is a sequence of objects that supports various methods which can be pipelined to produce result.

A Stream is not a data structure instead it takes input from the collections or arrays, Stream don't change original data structure; it produces the result as per the methods from source. Stream is lazy and evaluates code only when required. Stream has two types of operations

- Intermediate operations
- Terminal operations



Intermediate Operations

An intermediate operation is a lazy operation, which means that these operations are not executed immediately. Instead, they are executed only when the execution reaches a terminal operation. This lazy nature allows for optimization and efficiency when processing streams.

Let's look at some of the most popular intermediate operations used in streams:

1. Map:

The Map operation is used to return a stream consisting of the results of applying the given function passed in as parameter to the elements of the stream

2. Filter:

The Filter operation is used to return a stream of elements which satisfies the given predicate passed in as a parameter to the operation

3. Limit:

The limit operation restricts the number of elements in the stream to a specified size. This is particularly useful when you need to process only a portion of the stream.

4. Distinct:

The distinct operation removes duplicates from the stream, ensuring that each element appears only once in the resulting stream.

Terminal Operations

A Terminal operation is Eager operation that is to say, they start the execution of all previous lazy operations presented in the stream. Terminal operation either return concrete types or produce a result list. i.e., a reduce operation is returns single value by reducing the results into a single sum. A collect operation will create a new list of results, here are the most popular terminal operations used in the stream is

1. Collect :

Collect is used to return the list, map or set of results of intermediate operations performed on stream

2. ForEach :

ForEach is used to iterate through every element of stream after completing the execution of intermediate operation

3. Reduce :

Reduce is used to return the single value by cumulating the elements of stream after execution of intermediate operation

Okay, Now lets dive into practical with Stream API

Problem : Java program to print all the elements of list by multiplying with two ?

Traditional Solution

```
import java.util.*;
public class example {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        for (int i = 0; i < numbers.size(); i++) {
            int multiplied = numbers.get(i) * 2;
            System.out.println(multiplied);
        }
    }
}
```

Stream API Solution

```
import java.util.*;
import java.util.stream.*;
public class example {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        numbers.stream().map(n -> n * 2).forEach(System.out::println);
    }
}
```

Note: here by using **method reference** syntax i printed the data using system.out::println

Problem : Java program to print strings of an array which started with a specific keyword, print all strings in uppercase

Traditional Solution

```
import java.util.*;
import java.util.stream.*;
public class example {
    public static void main(String[] args) {
        String[] strings = {"Sri Krishna", "Balaram", "Jagannath", "Sri rama", "Kalki"};
        for (String str : strings) {
            if (str.startsWith("Sri"))
            {
                System.out.println(str.toUpperCase());
            }
        }
    }
}
```

Stream API Solution

```
import java.util.*;
import java.util.stream.*;
public class example {
    public static void main(String[] args) {
        String[] strings = {"Sri Krishna", "Balaram", "Jagannath", "Sri rama", "Kalki"};
        Arrays.stream(strings)
            .filter(s -> s.startsWith("Sri"))
            .map(String::toUpperCase)
            .forEach(System.out::println);
    }
}
```

Problem : Java program to print sum of even numbers from the list

Traditional Solution

```
import java.util.*;
import java.util.stream.*;
public class example {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
        int sum = 0;
        for (int number : numbers) {
            if (number % 2 == 0) {
                sum += number;
            }
        }
        System.out.println("Sum of even numbers: " + sum);
    }
}
```

Stream API Solution

```
import java.util.*;
import java.util.stream.*;

public class example {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        int sum = numbers.stream()
            .filter(number -> number % 2 == 0)
            .sum();

        System.out.println("Sum of even numbers: " + sum);
    }
}
```

Note: here we can use sum method to calculate the sum of all elements, Instead of sum we can use reduce method also to calculate sum as `.reduce(0,(i,j)->i+j);`

Problem : Java program to print all even numbers between 1 to 50

Traditional Solution

```
import java.util.*;
import java.util.stream.*;

public class example {
    public static void main(String[] args) {
        List<Integer> evenNumbers = new ArrayList<>();
        for (int i = 1; i <= 50; i++) {
            if (i % 2 == 0) {
                evenNumbers.add(i);
            }
        }
        System.out.println(evenNumbers);
    }
}
```

Stream API Solution

```
import java.util.*;
import java.util.stream.*;
import java.util.stream.Collectors;

public class example {
    public static void main(String[] args) {
        List<Integer> evenNumbers =
            Stream.iterate(1, i -> i + 1)
                .limit(50)
                .filter(i -> i % 2 == 0)
                .collect(Collectors.toList());
        System.out.println(evenNumbers);
    }
}
```

Traditional Solution

```
import java.util.*;
import java.util.stream.*;
import java.util.stream.Collectors;import java.util.stream.Collectors;
public class example {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 4, 5, 6, 6, 7, 8, 8);
        List<Integer> uniqueNumbers = new ArrayList<>();

        for (Integer number : numbers) {
            if (!uniqueNumbers.contains(number)) {
                uniqueNumbers.add(number);
            }
        }
        System.out.println(uniqueNumbers);
    }
}
```

Stream API Solution

```
import java.util.*;
import java.util.stream.*;
import java.util.stream.Collectors;import java.util.stream.Collectors;
public class example {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 4, 5, 6, 6, 7, 8, 8);
        List<Integer> distinctNumbers = numbers.stream()
                                                .distinct()
                                                .collect(Collectors.toList());

        System.out.println(distinctNumbers);
    }
}
```

Practice Problems

- Java program to list of divisible of 5 between 1 to 100 without using loops
- Java program to calculate the sum of all even, odd numbers in a list using streams
- Java program to count the frequency of each word (ignoring case) and return a map with words as keys and their frequencies as values
- Java program to find the longest string in the list. If there are multiple strings with the same maximum length,
- Java program to print the total salary of the employees grouped by department using the Stream API